

Deep Insights Engine — Build Guide

To the implementing agent: Build this in the order given in §18. Each layer has an acceptance test — do not proceed to the next layer until it passes. Every algorithm below cites the paper it comes from; if you are unsure of a detail, read that paper rather than improvising. The invariant in §0 is not negotiable and must not be relaxed for convenience.

What you are building: an engine that answers any question about any registered dataset with **10 insights, each 3 substantive paragraphs**, every number verifiable, in **under 8 minutes** using **fewer than 30 LLM calls**.

Stack: Google ADK · DuckDB · SQLite (source) · `semantica` (semantic layer) · Gemini Flash + Pro.

0. The invariant

The LLM proposes. Only SQL asserts.

- Deterministic components are tool-free `BaseAgent` subclasses or plain functions. **Zero LLM calls inside them.**
- LLM components receive compressed signal JSON, **≤800 tokens**, never result rows.
- A final validator drops any insight containing a numeral absent from its evidence bundle.

Test of correctness: run the same question twice. `seed_segments`, `dig_paths`, `decomposition`, `attribution`, `effect_size`, `p_value` must be **byte-identical**. Only prose and explored angles may differ. This is a CI test, not an aspiration.

1. Repository layout

```
insights/
├─ config/
│   ├── semantic.yaml           # metric/dimension extensions (§2.2)
│   └─ thresholds.yaml         # every tunable constant, one place
├─ core/
│   └─ adapter.py              # semantica wrapper (§2)
```

```

|   ├── workspace.py           # DuckDB session mgmt ($3)
|   ├── macros.sql             # DuckDB macro library ($4)
|   ├── extract.py             # base + cube ($5)
|   ├── miners/                # 20 miners, one file each ($6)
|   |   ├── __init__.py        # registry
|   |   ├── m01_level.py ... m20_forecast_gap.py
|   ├── rca.py                 # vendored RiskLoc adapter ($7)
|   ├── rank.py                # hierarchical summarization ($9)
|   ├── dig.py                 # MCTS ($10)
|   ├── resolve.py             # rival adjudication ($11)
|   ├── score.py               # scoring, MMR, BH ($13)
|   └── verify.py              # numeric binding ($13)
└── agents/
    ├── prompts.py             # ALL prompts, nowhere else
    ├── deterministic.py        # BaseAgent subclasses
    ├── llm_agents.py          # LlmAgent definitions
    └── root.py                 # topology ($14)
└── eval/
    ├── inject.py              # synthetic anomaly injection
    ├── cases.yaml             # 40 golden cases
    └── run_eval.py
└── vendor/riskloc/            # git subtree of shaido987/riskloc

```

Rule: no prompt string outside `agents/prompts.py` . No magic number outside `config/thresholds.yaml` .

2. LAYER 0 — Semantic adapter

Provenance: Cube’s “agent selects from certified definitions, never writes raw SQL”; Wren AI’s MDL grounding argument.

2.1 Required interface

Wrap `semantica` behind exactly this. If `semantica` lacks a method, implement it in the adapter.

```

class SemanticAdapter(Protocol):
    def list_metrics(self) -> list[MetricDef]: ...
    def get_dimensions(self, metric: str) -> list[DimensionDef]: ...
    def compile_sql(self, req: MetricRequest) -> str: ...
    def validate(self, req: MetricRequest) -> list[str]: ... # [] means valid

```

2.2 Four extensions you MUST add

Most semantic layers lack these. Put them in `config/semantic.yaml` , merged over whatever `semantica` returns.

```
metrics:
  conversion_rate:
    kind: ratio                                # additive | ratio | distinct_count
    numerator_sql: "SUM(converted)"
    denominator_sql: "COUNT(*)"              # REQUIRED when kind == ratio
    direction: up_good                        # up_good | down_good | neutral
    unit: "pp"
    good_threshold: 0.08

dimensions:
  city:
    hierarchy: [country, region, city]        # REQUIRED — Layer 9 cannot work without
    cardinality: 1840
    drillable: true
    bucket_sql: null
  customer_id:
    cardinality: 2400000
    drillable: false                          # excluded from lattice
    bucket_sql: "ntile(20) OVER (ORDER BY lifetime_value)"
```

Why each matters:

1. **hierarchy** — Layer 9 (hierarchical summarization) operates on a cartesian product of *trees*. Without declared parent–child chains there is no tree and no algorithm. Flat dimensions get a single-level tree: `ALL → value` .
2. **kind + separate numerator/denominator** — attributing a ratio metric as if it were additive produces confidently wrong root causes. This is the most common failure mode in commercial BI insight tools. See §7.2.
3. **direction** — determines whether a movement is good or bad. Without it every recommendation is generic filler.
4. **cardinality cap** — any dimension above 200 distinct values is excluded from the lattice unless it has `bucket_sql` . High-cardinality IDs flood the miners with noise and blow up the cube combinatorially. Power BI's own guidance: analyze "Customer Segment", never "Customer ID".

2.3 SQL guard

Every compiled string passes through this before execution:

```
FORBIDDEN = r"\b(DELETE|UPDATE|INSERT|DROP|ATTACH|DETACH|COPY|INSTALL|LOAD|PRAGMA
```

```
def guard(sql: str) -> None:
    if sql.count(";") > 1:
        raise Unsafe("single statement only")
    if not re.match(r"^\s*(SELECT|WITH|CREATE (OR REPLACE )?(TEMP )?TABLE \w+ AS)
        raise Unsafe("must be SELECT/WITH/CTAS")
    if re.search(FORBIDDEN, sql, re.I):
        raise Unsafe("forbidden keyword")
    if not re.search(r"\bWHERE\b", sql, re.I):
        raise Unsafe("time filter required")
```

Plus a watchdog thread calling `con.interrupt()` at `query_timeout_s` (default 20).

Acceptance: `adapter.validate()` rejects an unknown metric name, an unknown dimension, and a ratio metric missing its denominator. `guard()` rejects each of five crafted malicious strings.

3. LAYER 1 — Storage tiers

src	SQLite	system of record. READ_ONLY. Never written.
mirror	DuckDB / Parquet	columnar copy of hot facts. Watermark-refreshed.
ws	DuckDB (file)	per-session workspace. Deleted on completion.

3.1 Attach and mirror

```
ATTACH 'app.sqlite' AS src (TYPE sqlite, READ_ONLY);

-- one-time
CREATE TABLE mirror.fact AS SELECT * FROM src.fact;
-- or: COPY (SELECT * FROM src.fact) TO 'fact.parquet' (FORMAT parquet);

-- incremental (cron, or on src mtime change)
INSERT INTO mirror.fact
SELECT * FROM src.fact WHERE rowid > (SELECT max(rowid) FROM mirror.fact);
```

Decision rule: measure cold extraction (§5.1) against your largest fact table. If it exceeds 30 seconds, **the mirror is mandatory** — build it before writing any other code. DuckDB reads SQLite row-at-a-time through the `sqlite` extension; correct but not fast at scale. A 90-second scan through `sqlite_scanner` becomes ~2 seconds on the mirror.

3.2 Two traps that will bite you

Type affinity. SQLite is weakly typed; DuckDB deduces column types from declared types ("INT"→BIGINT, "TEXT"→VARCHAR) and *enforces* them. One stray string in an INTEGER column aborts the whole scan. Either validate at mirror-build time, or `SET GLOBAL sqlite_all_varchar=true` and cast explicitly in the extraction SQL. Decide now, not in production.

Double-linked SQLite. Never link two copies of the SQLite library into one process. If the ADK process also imports Python's `sqlite3`, route *all* SQLite access through DuckDB's ATTACH.

3.3 Workspace

```
def open_workspace(session_id: str) -> duckdb.DuckDBPyConnection:
    path = f"/tmp/insights/ws_{session_id}.duckdb"
    con = duckdb.connect(path)
    for stmt in [
        "SET memory_limit='8GB'",
        "SET threads=8",
        "SET temp_directory='/tmp/insights/spill'",
        "SET preserve_insertion_order=false",
    ]:
        con.execute(stmt)
    con.execute("ATTACH 'app.sqlite' AS src (TYPE sqlite, READ_ONLY)")
    con.execute(open("core/macros.sql").read())
    return con
```

Use a **file**, not `:memory:`, so DuckDB can spill to disk when a workload exceeds RAM. Store only the **path** in ADK state (state must be JSON-serializable). Sweep orphans older than one hour.

Acceptance: open a workspace, run `SELECT 1`, confirm spill directory is created, confirm the file is deleted on close.

4. LAYER 2 — DuckDB macro library

Why: DuckDB supports scalar *and table* macros. Defining the 20 miners as macros eliminates most string templating and makes them individually testable. Write this file once; everything downstream calls into it.

```
core/macros.sql :
```

```

-- === scalar helpers ===
CREATE OR REPLACE MACRO safe_div(a, b) AS
  CASE WHEN b IS NULL OR b = 0 THEN NULL ELSE a::DOUBLE / b END;

CREATE OR REPLACE MACRO pct_change(cur, prv) AS safe_div(cur - prv, abs(prv));

-- Jensen-Shannon divergence of two shares – Adtributor "surprise"
CREATE OR REPLACE MACRO js_div(p, q) AS
  0.5 * ( CASE WHEN p > 0 THEN p * ln(safe_div(2*p, p+q)) ELSE 0 END
    + CASE WHEN q > 0 THEN q * ln(safe_div(2*q, p+q)) ELSE 0 END );

-- two-proportion z statistic
CREATE OR REPLACE MACRO z_prop(x1, n1, x2, n2) AS
  safe_div( safe_div(x1,n1) - safe_div(x2,n2),
    sqrt( safe_div(x1+x2, n1+n2) * (1 - safe_div(x1+x2, n1+n2))
      * (safe_div(1,n1) + safe_div(1,n2)) ) );

-- epidemiological risk ratio – MacroBase/DIFF
CREATE OR REPLACE MACRO risk_ratio(a, b, c, d) AS
  safe_div( safe_div(a, a+b), safe_div(c, c+d) );

-- robust z via MAD
CREATE OR REPLACE MACRO robust_z(x, med, mad) AS safe_div(0.6745 * (x - med), mad

-- Herfindahl index component
CREATE OR REPLACE MACRO hhi_term(share) AS share * share;

```

Table macros follow in §6, one per miner.

Acceptance: `SELECT js_div(0.5, 0.5)` returns 0; `SELECT js_div(0.9, 0.1)` returns a positive value; `safe_div(1, 0)` returns NULL, not an error.

5. LAYER 3 — Extract and cube

5.1 Base extraction — the ONLY read of source data

```

CREATE OR REPLACE TABLE ws.base AS
SELECT
  {dim_list},
  date_trunc('{grain}', {time_col}) AS d,
  SUM({numerator_sql}) AS num,
  SUM({denominator_sql}) AS den,      -- literal 1 for additive metrics
  COUNT(*) AS n

```

```

FROM {source_ref}                                -- mirror.fact preferred
WHERE {time_col} >= $p0 AND {time_col} < $c1
    {extra_filters}
GROUP BY ALL;

```

Non-negotiable: always carry `num` **and** `den` separately, even for additive metrics (`den = 1`). Ratio attribution is impossible to reconstruct later.

`GROUP BY ALL` is DuckDB shorthand — use it. Result is typically 10k–500k rows regardless of source size, which is why everything downstream is fast.

Cache key: `(fact, filters_hash, period, dims_hash, source_mtime)` .

`os.path.getmtime()` on the SQLite file is a free, exact watermark. Follow-up questions in a conversation usually share scope and skip extraction entirely.

5.2 The cube — “see it in all directions”

```

CREATE OR REPLACE TABLE ws.cube AS
SELECT
    GROUPING_ID({dims}) AS cuboid,
    {dims},
    SUM(num) FILTER (WHERE d >= $c0) AS num_cur,
    SUM(num) FILTER (WHERE d <  $c0) AS num_prv,
    SUM(den) FILTER (WHERE d >= $c0) AS den_cur,
    SUM(den) FILTER (WHERE d <  $c0) AS den_prv,
    SUM(n)
    AS n
FROM ws.base
GROUP BY GROUPING SETS ( (), {one_d_sets}, {selected_pairs} )
HAVING SUM(n) >= $min_support;

```

Pair selection algorithm — never emit the full CUBE (2^n cuboids kills the query):

```

def select_pairs(dims, card, budget=2000, max_pairs=8):
    cands = [(a, b) for a, b in combinations(dims, 2)
              if card[a] * card[b] <= budget]
    # prefer pairs whose dims are individually informative
    cands.sort(key=lambda p: -(fact_score[p[0]] + fact_score[p[1]]))
    return cands[:max_pairs]

```

3-D cuboids are computed on demand only, when MCTS (§10) requests them.

5.3 Long-format view

Miners are easier to write against long format:

```
CREATE OR REPLACE VIEW ws.cube_long AS
UNPIVOT (SELECT * FROM ws.cube WHERE cuboid IN ({one_d_cuboid_ids}))
ON {dims} INTO NAME dim VALUE dim_value;
```

5.4 Hierarchy table (needed by Layer 9)

```
CREATE OR REPLACE TABLE ws.hier (
  dim VARCHAR, node_id VARCHAR, parent_id VARCHAR,
  level INTEGER, label VARCHAR
);
```

Populate from `DimensionDef.hierarchy`. For `[country, region, city]` you get `geo:ALL → geo:country:US → geo:region:West → geo:city:SF`. Flat dims get `ALL → value`.

Acceptance: on a 5M-row table with 6 dimensions, `ws.base` builds in <10s warm and `ws.cube` in <3s. `SELECT SUM(num_cur) FROM ws.cube WHERE cuboid=0` equals the ungrouped total (roll-up consistency test).

6. LAYER 4 — The 20 miners

Each miner is a table macro plus a thin Python wrapper emitting `Fact` rows. Uniform output shape:

```
@dataclass
class Fact:
    fact_id: str; miner: str; segment: Segment
    magnitude: float; magnitude_pct: float
    p_value: float; coverage: float; n: int
    evidence_id: str
```

6.1 The catalogue

#	Miner	Algorithm	Source
1	level	share of total, Pareto point	QuickInsights
2	change	Δ , $\Delta\%$ vs prior	—
3	attribution	explanatory power + surprise	Adtributor , NSDI'14
4	mix_vs_rate	rate/mix/interaction split	index-number theory

5	trend	<code>regr_slope</code> + CI, t-test	QuickInsights
6	changepoint	CUSUM	—
7	seasonality	autocorrelation at period lag	QuickInsights
8	slice_outlier	robust z (MAD) vs peers	QuickInsights
9	time_outlier	residual vs rolling median	—
10	distribution	quantile shift	—
11	rank_move	<code>RANK()</code> delta, extractor chain	Tang et al., SIGMOD'17
12	commonness	corr to median pattern	MetaInsight, SIGMOD'21
13	cohort	risk ratio, APriori $d \leq 3$	DIFF/MacroBase, VLDB'19
14	correlation	lagged <code>corr()</code>	—
15	concentration	HHI / top-N share, and its Δ	—
16	emergence	slices new or vanished	—
17	benchmark	gap to best-in-class peer	—
18	volatility	stddev ratio cur/prv	—
19	threshold	crossings of declared limits	—
20	forecast_gap	actual vs seasonal-naive	Adtributor (forecast baseline)

Miners 15–20 are the ones most systems skip; they produce the “how did it find that?” reactions.

6.2 Miner #4 — mix vs rate (build this first)

Highest value per line of code in the entire system. It distinguishes two causes that look identical on any dashboard.

For $M = N/D$, with slice weight $w_j = D_j / D$:

$$\begin{aligned}
 \Delta M &= \sum w_j^0 \cdot (M_j^1 - M_j^0) && \leftarrow \text{RATE effect} \\
 &+ \sum (w_j^1 - w_j^0) \cdot M_j^0 && \leftarrow \text{MIX effect} \\
 &+ \sum (w_j^1 - w_j^0) (M_j^1 - M_j^0) && \leftarrow \text{INTERACTION}
 \end{aligned}$$

```

CREATE OR REPLACE MACRO m_mix_rate(dim) AS TABLE
WITH tot AS (SELECT den_cur D1, den_prv D0 FROM ws.cube WHERE cuboid = 0),
    e AS (SELECT dim_value v,
        safe_div(den_cur, (SELECT D1 FROM tot)) w1,
        safe_div(den_prv, (SELECT D0 FROM tot)) w0,
        safe_div(num_cur, den_cur) m1,
        safe_div(num_prv, den_prv) m0
        FROM ws.cube_long WHERE dim = dim)
SELECT SUM(w0*(m1-m0)) AS rate_effect,
    SUM((w1-w0)*m0) AS mix_effect,
    SUM((w1-w0)*(m1-m0)) AS interaction;

```

- **Rate-dominant** → “the segments themselves got worse.”
- **Mix-dominant** → “no segment changed; volume shifted toward one that was always worse.”

Report as a headline number: *“78% of the 3.2pt drop is mix shift, not degradation.”* Also run attribution separately on numerator and denominator — often the story is “the denominator collapsed,” a completely different conversation.

6.3 Miner #3 — attribution (Adtributor)

Source: Bhagwan et al., *Adtributor: Revenue Debugging in Advertising Systems*, NSDI 2014. Reported >95% accuracy and order-of-magnitude reduction in troubleshooting time.

For dimension i , element j , with total actual A and forecast/prior F :

```

explanatory power    EP_ij = (A_ij - F_ij) / (A - F)
surprise              S_ij  = js_div(F_ij/F, A_ij/A)

```

Per dimension:

1. Sort elements by **surprise** descending.
2. Accumulate until cumulative $|EP| \geq T_{EP}$ (default 0.67).
3. Reject elements with surprise $< T_{EEP}$ (default 0.1).
4. Dimension score = Σ surprise over the chosen set.
5. Rank dimensions; report top 1–2.

Surprise is the part everyone skips, and it is what stops the algorithm naming your biggest segment every single time. A huge segment that moved proportionally with everything else has high EP and near-zero surprise — correctly boring.

Succinctness is the tiebreak: an explanation naming 2 elements beats one naming 9 at equal EP. Penalize by set size.

```
CREATE OR REPLACE MACRO m_attribution(t_ep, t_eep) AS TABLE
WITH tot AS (SELECT num_cur T1, num_prv T0 FROM ws.cube WHERE cuboid = 0),
  s AS (
    SELECT dim, dim_value,
           safe_div(num_cur-num_prv, (SELECT T1-T0 FROM tot)) AS ep,
           js_div(safe_div(num_prv, (SELECT T0 FROM tot)),
                 safe_div(num_cur, (SELECT T1 FROM tot))) AS surprise
    FROM ws.cube_long)
SELECT *, SUM(abs(ep)) OVER (PARTITION BY dim ORDER BY surprise DESC) AS cum_ep
FROM s
WHERE surprise >= t_eep
QUALIFY cum_ep <= t_ep
       OR row_number() OVER (PARTITION BY dim ORDER BY surprise DESC) = 1;
```

Known limitation: plain Adtributor only localizes first-layer (1-D) cuboids. When the best 1-D explanation's cumulative EP falls below 0.6, escalate to the RCA suite (§7).

6.4 Miner #13 — cohort risk ratio (DIFF)

Source: Abuzaid et al., *DIFF: A Relational Interface for Large-Scale Data Explanation*, VLDB 2019. A reference implementation exists in under 300 lines of Python generating SQL.

Given an outlier relation and an inlier relation with matching schemas, for predicate ϕ :

```
a = outliers matching  $\phi$       b = inliers matching  $\phi$ 
c = outliers not matching  $\phi$   d = inliers not matching  $\phi$ 
support      = a / (a + c)
risk_ratio = risk_ratio(a, b, c, d)
```

Algorithm: APriori-style. Score all 1-attribute predicates; prune below `min_support` (0.02) or `min_rr` (2.0); join survivors into 2-attribute combinations; repeat to depth 3. Support is anti-monotone so pruning is sound and it stays fast. Bin numeric columns into deciles first.

Output reads: *"Failed transactions were 8.4× more likely to be mobile × APAC × first-time-user. This combination covers 31% of failures but 4% of successes."* No dashboard produces this class of finding.

6.5 Miner #12 — commonness vs exception (MetaInsight)

Source: Ma et al., *MetaInsight*, SIGMOD 2021. Groups patterns into Homogeneous Data Patterns, splits into commonness and exceptions. Scoring is impact × conciseness, with a

regularization term penalizing cases with no exceptions — a pattern where everything agrees is true but not actionable.

Naive pairwise similarity is $O(n^2)$. Use median-pattern correlation instead:

```
CREATE OR REPLACE MACRO m_commonness(dim) AS TABLE
WITH raw AS (SELECT dim_value v, d, safe_div(SUM(num),SUM(den)) m
             FROM ws.base GROUP BY ALL),
    norm AS (SELECT v, d,
                   safe_div(m - avg(m) OVER (PARTITION BY v),
                             stddev(m) OVER (PARTITION BY v)) AS z
             FROM raw),
    med AS (SELECT d, median(z) mz FROM norm GROUP BY d)
SELECT n.v, corr(n.z, m.mz) AS fit,
       corr(n.z, m.mz) >= 0.5 AS is_commonness
FROM norm n JOIN med m USING (d) GROUP BY n.v;
```

This converts a list of facts into an argument:

Nine of eleven regions peaked in March and declined through Q2 — a consistent seasonal pattern. Two did not: Region D peaked in June, and Region G showed no seasonality. Region D's divergence accounts for 40% of Q2 variance.

Bake this shape into the narrator template whenever the signal type is

`commonness_exception`.

6.6 Miner #11 — extractor chaining

Source: Tang et al., SIGMOD 2017. Their formulation: an insight is an interesting observation derived from aggregation in *multiple steps*. Raw sales by brand shows nothing; yearly increase shows something; the **rank of yearly increase among brands** is the real insight.

Support chains of 2–3 steps: `aggregate → delta → rank → delta-of-rank`.

```
WITH agg AS (SELECT v, period, SUM(num) x FROM ws.base GROUP BY ALL),
    d AS (SELECT *, x - LAG(x) OVER (PARTITION BY v ORDER BY period) delta FROM agg),
    r AS (SELECT *, RANK() OVER (PARTITION BY period ORDER BY delta DESC) rk FROM d)
SELECT *, rk - LAG(rk) OVER (PARTITION BY v ORDER BY period) AS rank_move FROM r;
```

- Depth 1: *“Region B did \$4.2M.”* — chart caption.
- Depth 3: *“Region B climbed from 7th to 2nd in growth over two quarters, the largest rank move in the portfolio.”* — analyst.

Same data. The chaining is the difference.

Acceptance for Layer 4: each miner runs standalone against a fixture cube in <200ms; each returns the expected `Fact` for a hand-computed example; miners 3, 4, 12, 13 have unit tests against worked examples from their source papers.

7. LAYER 5 — RCA suite (vendored)

Do not reimplement any of these. Vendor `github.com/shaido987/riskloc` as a git subtree. It implements **RiskLoc**, **AutoRoot**, **RobustSpot**, **Squeeze**, **HotSpot**, and **Adtributor (plain and recursive)**, selectable by name.

7.1 Why three algorithms, not one

Algorithm	Strength	Weakness
Adtributor	fast, robust on 1-D	first-layer cuboids only
Squeeze	bottom-up then top-down, generalized ripple effect	assumes root causes cluster in one cuboid
RiskLoc	2-way partitioning + weighted risk score; best F1 on hard cases (up to 57% gain over second-best at comparable runtime)	slower than Adtributor

Run all three. Agreement is a confidence signal. Where all three converge → high confidence. Where they disagree → *that disagreement is itself narratable*: “the driver is concentrated in one channel by one method, distributed across three regions by another.” Feed the disagreement to the narrator as a caveat.

Reference point for feasibility: HotSpot reduced localization from over an hour of manual effort to under 20 seconds. Your budget is 15s for all three.

7.2 Ratio metrics — the critical caveat

Source: *CMMD: Cross-Metric Multi-Dimensional Root Cause Analysis*, arXiv 2203.16280.

Most business metrics are ratios (conversion rate, error rate, margin %, cost per unit). CMMD’s worked example is precisely the failure you must avoid: a segment with a huge conversion-rate change was flagged by both Adtributor and HotSpot, but its small volume made it irrelevant; CMMD correctly identified the segment whose 43% drop in conversions actually drove the 20% aggregate drop.

Implementation: when `MetricDef.kind == "ratio"`, run RCA on the **numerator** and **denominator** separately, then reconcile using the mix/rate decomposition from §6.2. Never run RCA directly on the ratio.

```
def run_rca(con, metric: MetricDef) -> list[Fact]:
    df = con.sql(RCA_INPUT_SQL).df()          # cuboid table only — small, pandas
    out = []
    if metric.kind == "ratio":
        for part in ("num", "den"):
            for algo in ("adtributor", "squeeze", "riskloc"):
                out += to_facts(riskloc.run(df, value_col=part, algorithm=algo),
                               source=f"{algo}:{part}")
            out += reconcile_ratio(out, mix_rate_result)
    else:
        for algo in ("adtributor", "squeeze", "riskloc"):
            out += to_facts(riskloc.run(df, algorithm=algo), source=algo)
    return dedupe_by_segment(out)
```

The input is the **cuboid table**, not the fact table — small enough for pandas even when your source is not.

Acceptance: on an injected single-dimension anomaly, all three algorithms name the correct dimension and element. On an injected 2-D anomaly, Squeeze and RiskLoc find it; Adtributor may not (expected, documented).

8. LAYER 6 — Debate (divergent → convergent)

Source: Liu et al., *DataSage*, arXiv 2511.14299 — external knowledge retrieval, multi-role debating, multi-path reasoning. Reported +7.5% insight-level and +13.9% summary-level over the best baseline, with strongest gains on high-difficulty tasks. The Question Raising Module uses a **divergent-convergent** multi-role process.

LLM calls: 1 (role generation) + 5 (diverge, parallel) + 1 (converge) = 7.

8.1 Data-derived roles — do not hardcode

```
ROLE_GEN (Flash, temp 0.7, 1 call)
```

```
Columns and metrics available: {profile}
```

```
Emit 5 analyst ROLES whose perspectives would surface DIFFERENT findings in
THIS dataset specifically. Each: {role_name, focus, what_they_worry_about}.
```

The roles must be derived from the data, not generic. Shape examples only – do not copy: a fraud analyst for payment/chargeback columns, a reliability engineer for latency/error columns, a merchandiser for catalog/inventory columns.

JSON only.

A profile containing `refund_amount`, `chargeback_flag` should produce a fraud role; `latency_ms`, `error_code` should produce a reliability role. One extra Flash call, materially broader coverage.

8.2 Diverge — temperature 0.9, parallel

ROLE_CLAIM (Flash, temp 0.9, 5 calls in parallel)

You are: {role_name} – {focus}. You worry about: {what_they_worry_about}.

Question: {question}

Established facts: {facts_compressed} # ≤400 tokens

Propose 3-5 CLAIMS worth testing from your perspective.

A claim is a TESTABLE PROPOSITION, not an observation. "Revenue fell 12%" is a fact, not a claim. "The fall is budget reallocation rather than performance decay is a claim.

Each MUST include `falsified_if` – a concrete result that would disprove it.

A claim without a valid `falsified_if` will be discarded.

Do not restate an established fact. Propose a mechanism, a comparison, or a confo

JSON: [{claim, kind, test:{action, params}, falsified_if}]

kind ∈ mechanism | comparative | structural | temporal | counterfactual | confoun

The mandatory `falsified_if` is the single constraint that eliminates most of the vague output LLMs produce here.

8.3 Converge — temperature 0.2, one Pro call

CONVERGE (Pro, temp 0.2, 1 call)

{n} roles proposed these claims: {all_claims}

Established facts: {facts_compressed}

Produce AT MOST 12 claims. For each:

- MERGE near-duplicates across roles into a single claim
- DROP anything the established facts already answer (no query needed)
- DROP anything unanswerable with available dimensions
- DROP anything lacking a valid falsified_if
- RANK by expected information gain (0-1)

JSON: [{claim_id, claim, kind, test, falsified_if, expected_gain}]

The temperature asymmetry is the mechanism. Wide stochastic search at 0.9 across different framings; tight near-deterministic filtering at 0.2. Do not run both at the same temperature.

8.4 Adjudicate — deterministic, zero LLM calls

Compile each claim's `test` to SQL via the action templates; run all concurrently (`asyncio.gather` over separate DuckDB cursors); return `Verdict` objects.

```
@dataclass
class Verdict:
    claim_id: str
    status: Literal["CONFIRMED", "REJECTED", "PARTIAL", "INSUFFICIENT_DATA"]
    effect_size: float
    p_value: float
    evidence_ids: list[str]
```

Keep the REJECTED ones. “We checked whether this was seasonal — it wasn’t; the same period last year moved the opposite way” is a real insight, and it is what makes a reader trust the other nine. **Reserve 2 of the 10 final slots for high-value rejections.**

Acceptance: every surviving claim has a `falsified_if`; every claim compiles to valid SQL or is dropped with a logged reason; adjudication of 12 claims completes in <10s.

9. LAYER 7 — Hierarchical ranking

Source: Ruhl, Sundararajan, Yan, *Hierarchical Summarization of Metric Changes*, arXiv 1703.07795. Studies metrics on a cartesian product of trees; identifies a small set of **non-overlapping** segments accounting for a change; **deployed in Google AdWords** for advertiser campaign triage. Optimal for 2 dimensions, approximation ratio $\log^{(d-2)}(n+1)$ for $d \geq 3$. NP-hard in general.

This is the most important component in the system. It solves two problems at once.

9.1 What it fixes

Naive top-K by score gives you ten restatements of one finding at arbitrary granularity. This algorithm returns segments that are **non-overlapping** and **already at the correct depth**.

9.2 The rule

Weight each node by $|metric_cur - metric_prv|$. Then bottom-up:

- Children moved in the **same direction** → parent weight = sum of children → **take the parent**. The segment is coherent; report it whole.
- Children moved in **opposing directions** → parent weight is smaller than a child's → **take the children**. The parent masks offsetting movements; drill in.

That single comparison is your entire granularity policy. It is why the algorithm "mimics the operation of a hierarchical organization of analysts."

Formally: choose overlap-free $S \subseteq V$, $|S| \leq k$, maximizing $\sum w(s)$.

9.3 Implementation — recursive CTE

```
WITH RECURSIVE nw AS (  
  SELECT h.node_id, h.parent_id, h.dim, h.level, h.label,  
         abs(c.num_cur - c.num_prv) AS wt,  
         sign(c.num_cur - c.num_prv) AS dir,  
         c.num_cur, c.num_prv, c.n  
  FROM ws.hier h  
  JOIN ws.cube_long c ON c.dim_value = h.label  
  WHERE h.level = (SELECT max(level) FROM ws.hier)  
  
  UNION ALL  
  
  SELECT p.node_id, p.parent_id, p.dim, p.level, p.label,  
         abs(SUM(ch.num_cur) - SUM(ch.num_prv)),  
         sign(SUM(ch.num_cur) - SUM(ch.num_prv)),  
         SUM(ch.num_cur), SUM(ch.num_prv), SUM(ch.n)  
  FROM ws.hier p JOIN nw ch ON ch.parent_id = p.node_id  
  GROUP BY ALL  
)  
SELECT n.*,  
       coalesce((SELECT SUM(c.wt) FROM nw c WHERE c.parent_id = n.node_id), 0) AS  
       n.wt >= coalesce((SELECT SUM(c.wt) FROM nw c WHERE c.parent_id = n.node_id  
                          AS take_self  
FROM nw n  
WHERE take_self  
ORDER BY wt DESC;
```

Then enforce overlap-freedom in Python:

```
def select_overlap_free(candidates, k=10):
    chosen = []
    for c in sorted(candidates, key=lambda x: -x.weight_final):
        if any(is_ancestor(c, s) or is_ancestor(s, c) for s in chosen):
            continue
        chosen.append(c)
        if len(chosen) == k:
            break
    return chosen
```

9.4 Weight refinements

Raw $|\Delta|$ is not enough. Apply, in order:

```
w_significance = w_raw * (1 - p_value)          # Sisu's principle: statistical val
                                                # filters BEFORE ranking, not after
w_final = 0.6 * w_significance + 0.4 * best_fact_score[segment]
```

The blend lets miner types other than “change” (outlier, cohort, concentration) seed a slot. Without it you get ten change-insights and nothing else.

9.5 Slot allocation

```
7 slots  hierarchical picks (the algorithm above)
2 slots  high-value REJECTED claims from $8.4
1 slot   highest-scoring "free angle" confirmed claim
```

Acceptance: on a fixture with a known offsetting pattern (region A +100, region B –100, parent net 0), the algorithm selects A and B, not the parent. On a coherent pattern (A +50, B +50), it selects the parent. No two returned segments overlap.

10. LAYER 8 — MCTS dig

Sources: Totis et al., *AIDE: MCTS for Knowledge Discovery* (JP Morgan AI Research), arXiv 2510.00876 — MCTS chosen for extensibility in integrating additional pattern strategies. Independently, Shi et al., *Calliope* (IEEE VIS 2020) uses a logic-oriented MCTS to progressively generate data facts and order them, with information theory scoring fact importance and a reward function prioritizing meaningful insights.

Two independent groups converging on MCTS for insight search is a strong signal.

10.1 Why MCTS and not a loop

UCT gives exactly what a fixed-depth loop cannot: **exploit a promising branch downward to granular depth while still exploring unvisited dimensions**. This is where your three paragraphs come from.

10.2 Action set

```
ACTIONS = [
    "refine_dim",          # add one predicate → deeper granularity
    "decompose_metric",    # numerator vs denominator movement
    "split_time",          # locate changepoint within the window
    "compare_peers",       # benchmark against sibling segments
    "cohort_diff",         # risk ratio inside this segment
    "test_confound",       # control for another dimension, re-attribute
]
```

Each compiles to one templated SQL query against `ws.base`.

10.3 The loop

```
def dig(con, seed: Segment, budget_s=45, max_depth=4, c=1.4) -> DigNode:
    root = DigNode(segment=seed, action="seed", visits=1, reward=0.0)
    deadline = time.monotonic() + budget_s

    while time.monotonic() < deadline:
        # 1. SELECT — descend by UCT
        node = root
        while node.children and node.segment.level < max_depth:
            node = max(node.children, key=lambda ch:
                ch.reward / ch.visits
                + c * sqrt(log(node.visits) / ch.visits))

        # 2. EXPAND — one untried action
        action = next_untried(node)
        if action is None:
            if node is root: break
            mark_exhausted(node); continue

        child = apply_action(con, node, action)      # ← ONE DuckDB QUERY
        if child is None or child.n < MIN_SUPPORT:
            mark_dead(node, action); continue
        node.children.append(child)
```

```

# 3. SIMULATE — the query IS the evaluation; no rollout needed
child.reward = score(child, ancestors=path_to_root(node))

# 4. BACKPROPAGATE
for anc in path_to_root(child):
    anc.visits += 1
    anc.reward += child.reward

return best_path(root)

```

The rollout is a DuckDB query, not an LLM call. This is what makes 40+ iterations per tree affordable in 45 seconds. Each query hits `ws.base` (small, cached in memory) and returns in single-digit milliseconds. **Zero LLM calls in the normal path.**

10.4 Reward

```

def score(node, ancestors) -> float:
    magnitude      = abs(node.metric_cur - node.metric_prv) / global_change
    concentration  = node.contribution      # share of PARENT's delta this node exp
    significance   = 1 - node.p_value
    novelty        = 1 - max((similarity(node, a) for a in ancestors), default=0)
    support        = min(1.0, node.n / MIN_GOOD_N)
    return magnitude * concentration * significance * novelty * support

```

concentration is what drives depth. A child explaining 70% of its parent's delta is worth descending into; one explaining 12% is not. This is the term that produces a coherent path rather than a scattered tree.

10.5 Stopping

Per branch: `level == max_depth; n < min_support; contribution < 0.25`; no action clears the reward threshold. Per tree: time budget.

MCTS is anytime by construction — the tree always holds a best-path-so-far. This is exactly the property the budget gate (§14.4) needs: cut `budget_s` from 45 to 15 and you lose depth on the tail, never the headline.

10.6 Output — the raw material for three paragraphs

<code>paid_social</code>	-2.3pp	seed from Layer 7
<code>L × APAC</code>	-1.6pp	70% of parent
<code>L × mobile</code>	-1.4pp	88% of parent
<code>L × first_session</code>	-1.2pp	86% of parent, RR 8.4×

change_{point}: Aug 12 step change, not drift

You are not asking the LLM to write longer. You are giving it more to say.

10.7 LLM fallback (≤2 calls total, all trees)

Only when all six standard actions are exhausted **and** budget remains: one Flash call proposing a custom action for the top branch. It emits an action + params, never SQL. Compile, validate, execute.

Acceptance: on an injected 3-level anomaly, the dig path reaches the injected granularity in >70% of runs. 10 trees complete within 50s wall-clock. Two runs on identical data produce identical paths.

11. LAYER 9 — Rivals and resolution

LLM calls: 3 (batched, parallel). Skippable under budget pressure.

This is what separates a defensible paragraph from a plausible-sounding one.

11.1 Rival generation (Flash, temp 0.8, 3 batched calls)

For each observation below, propose 2-3 ALTERNATIVE explanations that would produce the SAME observation.

Prioritise: confounds, Simpson's paradox, composition artifacts, measurement/tracking changes, selection effects, coincident external events.

Each rival needs a DISCRIMINATING TEST — a query whose result differs depending on which explanation is true.

Observations: {top_8_dig_paths_with_mechanisms}

JSON: [{insight_id, rival, why_plausible, discriminating_test, distinguishes_how}]

11.2 Resolution — deterministic, zero LLM

Run the discriminating tests, then apply rules. **No LLM judgment here.**

Outcome	Action
all rivals rejected	confidence = high ; ruled-out rivals become narrative caveats

a rival confirmed with larger effect size	swap — it becomes the primary explanation
multiple survive	confidence = contested ; present both
original rejected	demote to a rejection-insight

11.3 What this buys you

Without rivals: *"Paid social caused the drop."*

With rivals:

Paid social accounts for 61% of the decline. The obvious reading is channel underperformance, but CTR held flat while impressions fell 34% — this is spend reduction, not performance decay. Two alternatives were tested and ruled out: the decline is not attributable to a mix shift toward lower-converting devices (device composition moved less than 2pp), and it is not a measurement artifact (the tracking change on Aug 14 postdates the inflection by nine days).

Cost: ~15 seconds. Highest return per second in the pipeline.

12. LAYER 10 — Narration

LLM calls: 3 (3–4 insights per call, parallel).

Structure source: Calliope-Net's **topic → evidence → explanation** pattern, which guarantees an inverted pyramid guiding readers to the most important information first — standard practice in data journalism.

NARRATE (Flash, temp 0.7, 3 parallel calls)

Write insights {ids}. For each you have: dig_path, decomposition, attribution, changepoint, cohort, rivals_tested, rivals_surviving, confidence, evidence bundle

STRUCTURE — exactly 3 paragraphs per insight:

¶1 WHAT + WHEN

Headline at the granularity where the dig path TERMINATED, not the seed level

Quantify: absolute change, % change, share of total movement.

State when it began (changepoint) and whether step change or gradual drift.

Most important fact in the FIRST sentence.

¶2 WHY + HOW

Walk the dig path downward, giving each level's contribution share.
State the rate/mix/interaction split explicitly.
Name rivals tested and ruled out, with the number that ruled each out.
If confidence is "contested", present both surviving explanations.

¶3 IMPACT + RECOMMENDATION

Impact in the metric's own unit AND as share of total.
One concrete action, owner function, expected effect, effort estimate.
Confidence level and caveats (sample size, data-quality flags, confounds).

HARD RULES

- Every numeral must appear in the evidence bundle. Invent nothing. Estimate nothing.
- Never round differently than the bundle.
- Metric direction is {direction}; describe movement as good or bad accordingly.
- Vary sentence structure across insights — they are read consecutively.
- No hedging beyond what the stated confidence warrants.
- No preamble. Start with the finding.

Evidence: {bundles}

Why batched: 10 separate parallel calls hit rate limits and waste prompt overhead; sequential is 10× too slow. 3–4 per call across 3 parallel calls is the sweet spot — one ~35-second step.

13. LAYER 11 — Scoring, selection, verification

13.1 Scoring

Source: QuickInsights (Ding et al., SIGMOD 2019) — unified formulation with a systematic mining framework. MetaInsight — impact × conciseness with an exceptions regularizer.

`score = significance × impact × unexpectedness × conciseness`

- **significance** — p-value against the null *for that pattern type*. Trend: t-test on regression slope. Outlier: tail probability. Proportion: two-proportion z.
- **impact** — the slice's share of the total measure. A signal in a slice worth 0.3% of the business is noise regardless of p-value.
- **unexpectedness** — deviation from *forecast or seasonal baseline*, not from zero. Compare like-for-like periods (same weekday, same month-of-year).

- **conciseness** — entropy-style penalty on predicate count.

13.2 Diversity selection (MMR)

Applied only to the free-angle and rejection slots — Layer 7 already guarantees non-overlap for the main 7.

```
def mmr_select(cands, k, lam=0.6):
    sel = []
    pool = sorted(cands, key=lambda c: -c.score)
    while len(sel) < k and pool:
        best = max(pool, key=lambda c:
            lam * c.score - (1-lam) * max((sim(c, s) for s in sel), default=0))
        sel.append(best); pool.remove(best)
    return sel
```

Similarity signature: (miner, dimension, value_set, time_window, direction) , with **dimension overlap weighted heaviest**.

13.3 Verification — deterministic, fail closed

1. **Numeric binding.** Extract every numeral and named entity from each narrative. Each must resolve to a value in that insight's evidence bundle after format normalization.
Drop the insight, not the run.
2. **Benjamini–Hochberg** across all candidate facts at $\alpha = 0.05$. You tested hundreds of slices; without correction roughly 5% of ten insights are noise presented as finding.
3. **Contradiction check.** Two insights asserting opposite directions on overlapping segments → keep the higher-scoring one.
4. **Direction check.** Recommendation must align with `MetricDef.direction`.
5. **Log every drop with its reason.** That log is how you tune thresholds.

Numeric fidelity below 1.00 is a blocking bug, not a tuning issue.

14. LAYER 12 — ADK wiring

14.1 Constraints you must respect

- **ParallelAgent** modifies `InvocationContext.branch` per child
(`ParentBranch.ChildName`) but **all parallel children share the same `session.state`**.
Write results under distinct keys.

- **Known ADK bug:** an `LlmAgent` *with tools* invoked inside a custom agent's `_run_async_impl` can lose context after the tool response and loop on the same tool. **Therefore: deterministic agents are tool-free `BaseAgent` s that do their own work; tool-using `LlmAgent` s sit only directly under workflow agents.**
- **State prefixes:** `temp:` is shared across the whole invocation chain (same invocation id) — use it for the workspace path. `user:` and `app:` persist. Keep values JSON-serializable and shallow.
- Update state via `output_key`, `EventActions.state_delta`, or a context object. Direct `session.state` writes are for **reading** only.
- `LoopAgent` stops at `max_iterations` **or** when a sub-agent yields an event with `escalate=True`.
- **An agent without `output_key` writes nothing, and the next agent's {placeholder} silently resolves empty.** This is the most common wiring bug — check it first when something is mysteriously blank.

14.2 Topology

```
from google.adk.agents import (LlmAgent, SequentialAgent, ParallelAgent, BaseAgent)
from google.adk.events import Event, EventActions
from google.genai import types
```

```
FAST, SMART = "gemini-flash", "gemini-pro"
```

```
def cfg(t): return types.GenerateContentConfig(temperature=t)
```

```
# — PHASE 1 — 2 LLM calls, ends turn —
```

```
clarify_phase = SequentialAgent(name="p1", sub_agents=[
    LlmAgent("resolve", model=FAST, generate_content_config=cfg(0.0),
            instruction=RESOLVE_PROMPT, output_key="resolution"),
    ProfileAgent("profile"), # BaseAgent — SUMMARIZE
    LlmAgent("clarify", model=FAST, generate_content_config=cfg(0.3),
            instruction=CLARIFY_PROMPT, output_key="clarifications"),
    SetPhase("set_phase", phase="generate"),
])
```

```
# — PHASE 2 — 16 LLM calls —
```

```
generate_phase = SequentialAgent(name="p2", sub_agents=[
    LlmAgent("plan", model=FAST, generate_content_config=cfg(0.0),
            instruction=PLAN_PROMPT, output_key="plan"), # 1

    ExtractAgent("extract"), # 0
```

```

CubeAgent("cube"), # 0
MineAgent("mine"), # 20 miners + RCA suite # 0

LlmAgent("role_gen", model=FAST, generate_content_config=cfg(0.7),
         instruction=ROLE_GEN_PROMPT, output_key="roles"), # 1
DynamicRoleFanout("diverge"), # builds ParallelAgent # 5
LlmAgent("converge", model=SMART, generate_content_config=cfg(0.2),
         instruction=CONVERGE_PROMPT, output_key="claims"), # 1
AdjudicateAgent("adjudicate"), # 0

BudgetGate("gate_1", floor_s=150),
RankAgent("rank", k=10), # recursive CTE # 0
DigFanout("dig", budget_s=45), # 10 MCTS in asyncio # ≤2
BudgetGate("gate_2", floor_s=70),

RivalFanout("rivals", batches=3), # 3
ResolveAgent("resolve_rivals"), # 0

ParallelAgent("tell", sub_agents=[
    LlmAgent(f"nar_{b}", model=FAST, generate_content_config=cfg(0.7),
             instruction=NARRATE_PROMPT, output_key=f"narr_{b}"),
    for b in range(3)]), # 3
VerifyAgent("verify"), # 0
])

class PhaseRouter(BaseAgent):
    async def _run_async_impl(self, ctx):
        target = clarify_phase if ctx.session.state.get("phase", "clarify") == "cl
                    else generate_phase
        async for ev in target.run_async(ctx):
            yield ev

root_agent = PhaseRouter(name="deep_insights")

```

Total: 18 LLM calls. 12 spare for retries.

14.3 Deterministic agent template

```

class DigFanout(BaseAgent):
    budget_s: int = 45

    async def _run_async_impl(self, ctx):
        ws_path = ctx.session.state["temp:ws_path"]
        seeds = ctx.session.state["seed_segments"]
        budget = ctx.session.state.get("dig_budget_s", self.budget_s)

```

```

async def one(seed):
    con = duckdb.connect(ws_path, read_only=True)    # own cursor per task
    try:
        return await asyncio.to_thread(dig, con, seed, budget)
    finally:
        con.close()

paths = await asyncio.gather(*(one(s) for s in seeds))
yield Event(author=self.name, actions=EventActions(
    state_delta={"dig_paths": [p.to_dict() for p in paths]}))

```

One DuckDB connection per concurrent task. DuckDB handles multiple readers on one file in-process.

14.4 Budget gate

```

class BudgetGate(BaseAgent):
    floor_s: int

    async def _run_async_impl(self, ctx):
        left = ctx.session.state["deadline_ts"] - time.time()
        delta = {}
        if left < self.floor_s * 0.5:
            delta = {"skip_rivals": True, "dig_budget_s": 15, "narrate_batches":
        elif left < self.floor_s:
            delta = {"max_rivals": 3, "dig_budget_s": 25}
        yield Event(author=self.name, actions=EventActions(state_delta=delta))

```

Degrade, never overrun. Every phase is ranked, so cutting costs depth on the tail and never the headline.

15. Budget

Stage	Mode	Warm	Cold	LLM
resolve + profile + clarify	P/D/P	15s	15s	2
plan	P	4s	4s	1
extract <code>ws.base</code>	D	8s	60s	0
cube + 20 miners	D	7s	7s	0

RCA suite x3	D	15s	15s	0
role_gen + diverge + converge	P	22s	22s	7
adjudicate	D	10s	10s	0
hierarchical rank	D	4s	4s	0
MCTS dig x10	D	50s	50s	≤2
rivals + resolve	P/D	18s	18s	3
narrate x3	P	35s	35s	3
verify	D	4s	4s	0
TOTAL		3:12	4:04	18

Inside 5 min warm, 8 min cold, with two gates as the guarantee.

16. Evaluation harness

Source: InsightBench (Sahu et al., ICLR 2025) — built by injecting known trends into real tables and creating ground-truth analysis notebooks, evaluating the complete analytics process rather than a single query.

Build this at step 2 of the build order, not at the end. You cannot tune `T_EP`, `T_EEP`, `min_support`, `min_rr`, `τ`, `UCT_c`, or the reward weights without it.

16.1 Injection

```
INJECTIONS = [
    ("step_change", {"segment": {...}, "magnitude": -0.30, "at": "2026-08-12"}),
    ("mix_shift", {"from": {...}, "to": {...}, "weight_delta": 0.15}),
    ("trend_break", {"segment": {...}, "at": "..."}),
    ("cohort_effect", {"predicate": {...}, "risk_ratio": 6.0}),
    ("simpson", {...}), # aggregate up, every segment down
    ("deep_3level", {...}), # tests dig depth specifically
    ("seasonal_only", {...}), # NEGATIVE case — must be REJECTED
    ("pure_noise", {...}), # NEGATIVE case — must find nothing
]
```

Include negative cases. An engine that never says “no real change here” is useless and will

be trusted less, not more.

16.2 Targets

Metric	Target
Recall@10	> 0.85
Precision	> 0.80
Attribution accuracy (dimension and elements)	> 0.75
Depth accuracy (path reaches injected granularity)	> 0.70
Numeric fidelity	1.00
False-positive rate	< 0.10
p95 latency warm	< 300s
LLM calls per run	< 30

16.3 Determinism test (CI)

Run the same question twice. Assert byte-identical `seed_segments` , `dig_paths` , `decomposition` , `attribution` , `effect_size` , `p_value` . Prose and rival sets may differ.

17. Provenance map

Layer	Component	Source
0	certified definitions, never raw SQL	Cube; Wren AI MDL
0	cardinality cap	Power BI Key Influencers guidance
1	pre-aggregation for agent fan-out	Cube
3	GROUPING SETS lattice	Gray et al. data cube; DuckDB
4	insight scoring framework	QuickInsights, SIGMOD’19
	extractor chaining (compound	

4	insights)	Tang et al., SIGMOD'17
4	explanatory power + surprise + succinctness	Adtributor, NSDI'14
4	risk ratio / cohort explanation	DIFF + MacroBase, VLDB'19
4	commonness vs exception	MetaInsight, SIGMOD'21
4	rule notation (a, b, ★, n)	Smart Drill-Down, VLDB'15
5	RiskLoc / Squeeze / HotSpot / AutoRoot	<code>shaido987/riskloc</code>
5	ratio-metric handling	CMMD, arXiv 2203.16280
5	impact × statistical validity before ranking	Sisu (MacroBase commercialized)
6	divergent-convergent multi-role debate	DataSage, arXiv 2511.14299
6	analysis actions, not SQL	InsightPilot, EMNLP'23
7	hierarchical summarization, non-overlap	Ruhl/Sundararajan/Yan, arXiv 1703.07795 (Google AdWords)
8	MCTS for insight search	AIDE, arXiv 2510.00876 (JPMorgan); Calliope, IEEE VIS'20
10	topic → evidence → explanation	Calliope-Net; data journalism
12	≤3 clarifying questions	Wren AI
16	anomaly-injection evaluation	InsightBench, ICLR'25

18. Build order

Each step is independently shippable. **Do not skip the acceptance test.**

#	Step	Acceptance
1	Semantic adapter + hierarchy/direction/kind extensions	validate() rejects 3 malformed requests; every dim has a hierarchy

2	Eval harness with injection	40 cases load; injection is reproducible from a seed
3	Storage tiers + mirror decision	cold scan measured; mirror built if >30s
4	<code>ws.base</code> + <code>ws.cube</code> + macro library	roll-up consistency test passes; <10s warm
5	20 miners	each <200ms; miners 3/4/12/13 match worked examples from papers
6	RCA suite vendored	all 3 agree on a 1-D injection; Squeeze+RiskLoc find the 2-D one
7	Hierarchical ranking	offsetting → children; coherent → parent; zero overlap
8	MCTS dig	depth accuracy >0.70; identical paths across runs
9	3-paragraph narration from paths	numeric fidelity 1.00 on 40 cases
10	Rivals + resolve	≥1 rival ruled out on 80% of insights
11	Debate (roles → diverge → converge)	claims all have falsified_if
12	Budget gates + verify + determinism CI	p95 <300s; determinism test green

Steps 4–8 contain the intelligence. They would produce good insights with the LLM removed entirely. Steps 9–11 make it readable and conversational.

Definition of done:

- 10 insights, 3 paragraphs each, on any registered dataset
 - every numeral traceable to an `evidence_id`
 - p95 under 5 minutes warm, under 8 cold
 - fewer than 30 LLM calls per run
 - determinism test green in CI
 - recall@10 > 0.85 on the injected benchmark
-

19. Anti-patterns

Don't	Because
Let the LLM write raw SQL	Unauditable; hallucinates columns; fails governance
Send result rows to an LLM	Blows prompt, latency, and arithmetic accuracy
Take top-K by score	Ten restatements of one finding
Attribute ratio metrics as additive	Confidently wrong root causes (see CMMD)
Emit the full CUBE	2^n cuboids kills the query
Include high-cardinality IDs	Noise floods miners; performance collapses
Skip Benjamini–Hochberg	~5% of your insights are noise
Narrate sequentially or one-per-call	10× time, or rate limits
Fixed loop depth instead of MCTS	Shallow findings, no granularity
Hardcode analyst roles	Same five perspectives on every dataset
Reimplement Adtributor/Squeeze/RiskLoc	The reference repo has all six
Nest tool-using LlmAgents in custom agents	Known ADK context-loss bug
Prompt for “more detail”	Produces padding; dig paths produce analysis
Miss the deadline instead of degrading	MCTS is anytime — always have a best-so-far

20. References

Attribution / root cause

- Bhagwan et al., *Adtributor: Revenue Debugging in Advertising Systems*, NSDI 2014
- *Squeeze: Generic and Robust Localization of Multi-dimensional Root Causes*, ISSRE 2019
- Kalander, *RiskLoc*, arXiv 2205.10004 · repo: github.com/shaido987/riskloc

- *CMMD: Cross-Metric Multi-Dimensional Root Cause Analysis*, arXiv 2203.16280
- Ruhl, Sundararajan, Yan, *Hierarchical Summarization of Metric Changes*, arXiv 1703.07795

Insight mining

- Tang et al., *Extracting Top-K Insights from Multi-dimensional Data*, SIGMOD 2017
- Ding et al., *QuickInsights*, SIGMOD 2019
- Ma et al., *MetaInsight*, SIGMOD 2021
- Abuzaid et al., *DIFF: A Relational Interface for Large-Scale Data Explanation*, VLDB 2019
- Joglekar, Garcia-Molina, Parameswaran, *Smart Drill-Down*, VLDB 2015

Agents / search

- Ma et al., *InsightPilot*, EMNLP 2023
- Totis, Pozanco, Borrajo, *AIDE: MCTS for Knowledge Discovery*, arXiv 2510.00876
- Shi et al., *Calliope*, IEEE VIS 2020
- Liu et al., *DataSage*, arXiv 2511.14299
- Sahu et al., *InsightBench*, ICLR 2025

Docs

- DuckDB: `sql/dialect/friendly_sql` , `sql/statements/pivot` ,
`core_extensions/sqlite`
- ADK: `agents/custom-agents` , `sessions/state` , `context`